

CSE 3206 Software Engineering Sessional

Lab 3: Interpreter Design Pattern

Role-Based Access Control Rule Interpreter

Group 4 • Rajshahi University of Engineering & Technology (RUET)

1. Pattern Name

Interpreter Pattern

2. Category

Behavioral Design Pattern.

3. Intent

Represent the grammar of a small language using objects and interpret expressions according to that grammar. Here the language describes role-based access-control rules.

4. Problem Statement

An application may need to evaluate rules such as **ADMIN**, **ADMIN AND EDITOR**, **EDITOR OR MODERATOR**, and **ADMIN AND (EDITOR OR MODERATOR)**. Hard-coding every possible rule with separate conditional branches becomes difficult to maintain and extend as rules grow.

5. Motivation

The Interpreter Pattern separates rule representation from evaluation. Roles become terminal expressions, logical operators become non-terminal expressions, and the complete rule becomes an expression tree that can be evaluated recursively.

6. Pattern Structure

The abstract **Expression** declares **interpret(Context)**. **RoleExpression** is a terminal expression. **AndExpression** and **OrExpression** are non-terminal expressions. **Context** stores user roles. **Parser** builds the expression tree.

7. Class Responsibilities

Expression: common abstraction. **RoleExpression**: checks one role. **AndExpression**: combines two child expressions using AND. **OrExpression**: combines two child expressions using OR. **Context**: stores roles. **Parser**: converts a rule string to an expression tree. **Client**: supplies the context and evaluates the expression.

8. Code Implementation

Implemented in C++17 using an abstract Expression class, terminal/non-terminal expression classes, Context, and a small recursive-descent Parser. Smart pointers manage expression-tree ownership.

9. Execution Flow

The client creates a Context, the Parser converts a textual rule into an expression tree, and **interpret(Context)** recursively evaluates that tree to produce TRUE or FALSE.

10. Advantages

- Composable expression objects.
- Extensible expression types.
- Separation of rule representation and evaluation.

- Reusable expressions with different contexts.
- Clear object interaction.

11. Limitations

- Large grammars may create many classes.
- Fixed simple conditions may be easier with ordinary conditionals.
- Large expression trees add complexity and recursive evaluation overhead.

12. Real-life Applications

Interpreter-style designs are useful for access-control rules, filtering expressions, search conditions, configuration expressions, rule engines, and mathematical expression evaluators.

13. Industry Examples

Expression and rule evaluation is common in authorization systems, search/filter interfaces, configuration systems, workflow rules, and query processors. This project demonstrates the design approach rather than claiming a specific commercial product uses this exact class structure.

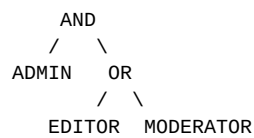
14. Demonstration

Select roles in the web interface and enter a rule. Example: ADMIN + EDITOR with **ADMIN AND (EDITOR OR MODERATOR)** produces ACCESS GRANTED. ADMIN alone with **EDITOR OR MODERATOR** produces ACCESS DENIED. The C++ implementation provides the main pattern demonstration.

15. Conclusion

The Interpreter Pattern provides a structured way to represent and evaluate a small grammar. The project demonstrates terminal and non-terminal expressions, Context, expression trees, recursive interpretation, testing, UML, and a visual web demonstration.

Expression Tree Example



ADMIN -> TRUE
EDITOR -> TRUE
MODERATOR -> FALSE

EDITOR OR MODERATOR -> TRUE
ADMIN AND TRUE -> TRUE
FINAL RESULT -> ACCESS GRANTED

Pattern-to-Class Mapping

Pattern Role	Project Class
Abstract Expression	Expression
Terminal Expression	RoleExpression
Non-Terminal Expression	AndExpression / OrExpression
Context	Context
Client / Parser	main.cpp / Parser